INSTALLATION / RASPBERRY PI 3B+

Setting up the Raspberry Pi with the Firebase backend

Step-by-step guide to install, start and verify local control

This guide installs the new Python backend on the Raspberry Pi, enables the SPI sensors and sets up automatic start as a systemd service. Firebase synchronizes status, schedule and commands; relay control and the active configuration stay on the Raspberry Pi. After a reboot, the backend re-reads the local config.json file even if the network is unavailable.

Before you begin

- Raspberry Pi 3B+ with Raspberry Pi OS and access to an account with sudo.
- SSH connection, or keyboard and screen for the commands on the Pi.
- Two MAX6675 modules with K-type thermocouples and two relay modules compatible with 3.3 V logic.
- API key, Realtime Database URL, email, password and UID of the dedicated Firebase user.
- The updated backend files on your PC. This procedure transfers them with SCP: no Git push is required.

Three operating rules

Electrical safety. Keep mains loads disconnected during the first check. The backend initializes the relays to OFF, but it may switch them on after the first temperature samples. Complete the wiring of mains-voltage circuits only with adequate protection and electrical competence.

Offline use. The service starts without waiting for an Internet connection. If Firebase is unreachable, control continues with the last local config.json and synchronization retries in the background.

Credentials. The systemd encrypted secret must be created on the Raspberry Pi that will run the service. Do not copy the encrypted file from Windows or from another Linux installation.

1 Connect sensors and relays

Disconnect power before wiring. The backend uses BCM numbering: GPIO17 and GPIO27 identify logical lines, while the physical numbers indicate the position on the 40-pin header.

Connections of the two MAX6675 modules

Module signal	Raspberry GPIO	Physical pin	Connection
VCC	—	1	3V3; check that the breakout is compatible with 3.3 V logic
GND	—	6	Common ground
SCK	`GPIO11 / SCLK`	23	Shared by both modules
SO	`GPIO9 / MISO`	21	Shared by both modules
CS of CH1	`GPIO8 / CE0`	24	Chip select of the first sensor
CS of CH2	`GPIO7 / CE1`	26	Chip select of the second sensor

Relay module inputs

Channel	BCM GPIO	Phys. pin	Module input
CH1	`GPIO17`	11	IN / SIG of relay module 1
CH2	`GPIO27`	13	IN / SIG of relay module 2

The relay driver must accept the 3.3 V logic level of the Raspberry Pi. Do not connect coils, loads or mains contacts directly to the GPIOs. The Pi's GPIO pins are not 5 V tolerant: power a breakout at 5 V only if its circuit guarantees safe levels towards the Raspberry.

The code opens SPI bus 0, device 0 for CH1 and device 1 for CH2. It drives GPIO17 and GPIO27 as relay outputs and uses active-high logic (RELAY_ACTIVE_LOW = False). If your module is active-low, do not change this setting without first verifying the behavior with the loads disconnected.

2 Prepare Raspberry Pi OS

01 Log in with the user that will run the service

In the example the user is pi and the project folder is /home/pi/rpi_thermostat. If your user has a different name, replace it in all paths and in the systemd file.

```
whoami
hostname -I
systemd --version | head -n 1
systemd-creds --version
```

systemd-creds and LoadCredentialEncrypted credentials are available from systemd 250. If the installed version is lower than 250, stop before the credentials step and upgrade Raspberry Pi OS to a supported version.

02 Update the index and install the Python packages

```
sudo apt update
sudo apt install -y python3 python3-requests python3-gpiozero python3-spidev
```

These packages provide Python, Firebase HTTPS requests, GPIO Zero and SPI access. Flask is not needed for the new backend.

03 Enable SPI

```
sudo raspi-config
```

In the menu select 3 Interface Options → I4 SPI → Yes, then Finish. Reboot the Pi if prompted. Raspberry Pi OS enables the interface and loads the SPI module at boot.

```
sudo reboot
# After the reboot
ls -l /dev/spidev0.0 /dev/spidev0.1
```

Both /dev/spidev0.0 and /dev/spidev0.1 must appear. If one is missing, recheck that SPI is enabled and reboot.

3 Transfer the backend and keep the schedule

Before starting the new process, stop any old backend: two processes must not control the same GPIOs. If you already had a config.json on the Raspberry Pi, keep it in the project folder; it contains the latest configuration and the local schedule.

Stop and back up the existing configuration

```
sudo systemctl list-units --type=service --all | grep -i thermostat
# If you find the old service, stop it using its actual name
sudo systemctl stop PREVIOUS_SERVICE_NAME
cd /home/pi/rpi_thermostat
test ! -f config.json || cp -a config.json "config.json.pre-firebase.$(date +%Y%m%d-%H%M%S).bak"
```

Replace PREVIOUS_SERVICE_NAME only if the service exists. If the old backend was started manually, stop that process before continuing. Do not copy a new config.json over the one already present.

Send the files from the Windows PC without Git

Open PowerShell in the local project folder. Replace the address 192.168.1.50 with the one shown by hostname -I; replace pi if you use a different account. SSH must be enabled on the Raspberry.

```
ssh pi@192.168.1.50 "mkdir -p /home/pi/rpi_thermostat"
scp .\rpi_thermostat_backend_main.py .\firebase_client.py .\setup_firebase_credentials.py
.\thermostat.service.example pi@192.168.1.50:/home/pi/rpi_thermostat/
```

The essential files are the backend, the Firebase client, the credentials script and the service example. The config.json file stays on the Pi. No changes are sent to Git.

```
# On the Raspberry Pi
cd /home/pi/rpi_thermostat
ls -l rpi_thermostat_backend_main.py firebase_client.py setup_firebase_credentials.py
thermostat.service.example
```

How config.json is recovered

On first start, if the file does not exist, the backend creates an initial configuration. On later starts it reads and normalizes config.json before beginning control. It saves changes using a temporary write and atomic rename. If the file is corrupted, it stops without replacing it with default values: keep the copy and fix it before restarting.

4 Configure the Firebase credentials

Use the same Linux account indicated by User= in the service (pi in the example). Do not run the script with sudo: the encrypted file must end up in the home of the intended user.

```
cd /home/pi/rpi_thermostat
python3 setup_firebase_credentials.py
```

Enter the Web API key, Realtime Database URL, email, password and UID of the Firebase user. The password is requested without echoing it on the terminal. The Web API key identifies the project; the RTDB rules and the dedicated account control access.

The script creates ~/.config/rpi_thermostat/firebase_config.cred, encrypted through systemd-creds. The plain text is written temporarily with restrictive permissions and then removed by the script; do not create or transfer a JSON file containing the password.

```
systemd-creds --version
stat -c '%a %n' ~/.config/rpi_thermostat ~/.config/rpi_thermostat/firebase_config.cred
```

Expected permissions are 700 on the folder and 600 on the .cred file. The systemd key is tied to the local installation: recreate the credential on the Pi after reinstalling the system or replacing the device.

5 Install the systemd service

```
sudo cp thermostat.service.example /etc/systemd/system/thermostat.service
sudo nano /etc/systemd/system/thermostat.service
```

Check and edit these values if you do not use the pi user or the example path:

Directive	Example value	Must match
User	pi	User that owns the encrypted credentials
WorkingDirectory	/home/pi/rpi_thermostat	Folder containing the Python files and config.json
ExecStart	/usr/bin/python3 /home/pi/rpi_thermostat/rpi_thermostat_backend_main.py	Actual interpreter and backend path
SupplementaryGroups	gpio spi	Groups that authorize GPIO and SPI
LoadCredentialEncrypted	/home/pi/.config/rpi_thermostat/firebase_config.cred	Path produced by the credentials setup

The example file does not wait for network-online.target: the service starts even without a network and the backend continues local control. systemd delivers the decrypted credential to the process in a read-only temporary directory.

```
sudo systemd-analyze verify /etc/systemd/system/thermostat.service
```

6 Start and verify the backend

For the first start, leave the load controlled by the relays disconnected. Check that the modules are wired correctly and that no relay can power a heating element or a dangerous load during the test.

```
sudo systemctl daemon-reload
sudo systemctl enable --now thermostat
sudo systemctl status thermostat --no-pager
```

The first command reloads the systemd units; the second enables start at every boot and starts the service immediately. The status must be active (running).

Read the logs

```
sudo journalctl -u thermostat -n 80 --no-pager
sudo journalctl -u thermostat -f
# Press Ctrl+C to exit the live log
```

Look for the backend start and the absence of SPI, GPIO or credentials errors. When Firebase is available, the thermostat/status node is updated every second. The frontend considers the Raspberry online when the timestamp is less than 10 seconds old.

Check the local file and the cloud

```
sudo systemctl is-enabled thermostat
sudo systemctl is-active thermostat
ls -l /home/pi/rpi_thermostat/config.json
cat /home/pi/rpi_thermostat/config.json
```

- The service answers enabled and active.
- config.json exists next to the backend. If it already existed, it still contains the previous setpoints and schedules.
- In the Firebase Console, thermostat/status has a recent updated_at and a consistent config_revision.
- The web frontend shows THERMOSTAT ONLINE within about 10 seconds of the status being published.

System time and schedule

The backend compares the schedule intervals with the local clock of the Raspberry Pi. Set the Europe/Rome time zone and check that the time is synchronized when the network is available:

```
sudo raspi-config
# 5 Localisation Options -> L2 Timezone -> Europe -> Rome
timedatectl status
```

config.json keeps the schedule even without power or network. Without an external RTC, however, the exact time after a long power outage followed by a start without network is not guaranteed: the schedule may be evaluated against a wrong time until the Pi synchronizes. To respect the schedule even in this condition, add a compatible RTC module.

Verify reboot without network

After checking the loads safely, reboot the Pi and check that thermostat is active. To test offline operation, remove only the network connection, without removing power from the control circuits: the frontend will show offline, while the backend will continue with config.json. Once the network is restored, Firebase synchronization restarts automatically.

7 Troubleshoot common problems

Symptom	Check	Action
systemd-creds missing or version < 250	systemd-creds --version	Upgrade Raspberry Pi OS to a version with systemd 250 or later.
The service does not start	sudo journalctl -u thermostat -n 80 --no-pager	Fix User, WorkingDirectory, ExecStart and the LoadCredentialEncrypted path.
Permission denied on GPIO/SPI	id pi; ls -l /dev/spidev0.*	Verify the gpio and spi groups and SupplementaryGroups; restart the service.
File /dev/spidev0.0 or .1 missing	ls -l /dev/spidev0.0 /dev/spidev0.1	Enable SPI with raspi-config and reboot the Pi.
Credential not readable	stat -c '%a %n' ~/.config/rpi_thermostat/firebase_config.cred	Create the .cred as the service user and update the path in the unit.
Frontend offline but service active	Open thermostat/status and check updated_at	Check Wi-Fi/Ethernet, the Firebase rules and the authentication logs. Cloud offline does not stop local control.
The backend stops on config.json	Read the error in the logs and keep the file	Restore a valid backup copy; do not delete config.json if it contains the schedule to be recovered.

Update the code without touching the schedules

For a new version, stop the service, copy the updated Python files into the same folder, then restart. Do not replace config.json and do not regenerate the .cred if the account and system are unchanged.

```
sudo systemctl stop thermostat
# From Windows: repeat the scp command from the previous page
sudo systemctl start thermostat
sudo systemctl status thermostat --no-pager
```

The config.json file stays on the local disk and is loaded before control starts. Keep the backup copy in a protected place until you have verified the service.

Enable the shutdown button (optional)

The backend invokes sudo shutdown now when it receives the web command. If your service user does not already have passwordless sudo authorization for this command, you can grant only shutdown instead of enabling general sudo.

```
command -v shutdown
sudo visudo -f /etc/sudoers.d/rpi-thermostat-shutdown
# In the file, replace pi and the path with the real values:
pi ALL=(root) NOPASSWD: /usr/sbin/shutdown now
sudo chown root:root /etc/sudoers.d/rpi-thermostat-shutdown
sudo chmod 0440 /etc/sudoers.d/rpi-thermostat-shutdown
sudo visudo -cf /etc/sudoers.d/rpi-thermostat-shutdown
```

If command -v shutdown returns a different path, use that path in the rule. This section is optional: local thermal control works even if you do not enable remote shutdown.

Technical references

Raspberry Pi Documentation · Enabling SPI · Accessed on October 27, 2026

Raspberry Pi Documentation · Time zone · Accessed on October 27, 2026

Raspberry Pi Documentation · GPIO pins and logic levels · Accessed on October 27, 2026

Raspberry Pi Documentation · RTC clock · Accessed on October 27, 2026

Analog Devices · MAX6675 datasheet · Accessed on October 27, 2026

systemd.exec · credentials for services · Accessed on October 27, 2026

systemd-creds · encrypt and supported versions · Accessed on October 27, 2026